Vacant Quarter Auction System

The complete code, explained for a beginner

Django 5.2 · Django Channels (WebSockets) · PostgreSQL · ReportLab

Training project. This is a proposed system built during BTech training at SAIL (Bokaro Steel Plant). It is **not** an official SAIL system. Every amount, rule and piece of sample data in it is a placeholder unless stated otherwise.

Generated October 2026 from the code in the repository at that time.

Contents

1. How to read this guide 4

2. The big picture: what the app does 5

2.1 The nine phases . . . 5

2.2 The people who use it . . . 5

2.3 The technology, in one sentence each . . . 5

3. Basic ideas you need first 7

3.1 What happens when you open a web page . . . 7

3.2 Django's pattern: Model - View - Template . . . 7

3.3 The ORM: talking to the database in Python . . . 7

3.4 Apps . . . 7

3.5 Forms and validation . . . 8

3.6 Settings and the .env file . . . 8

3.7 Transactions: all or nothing . . . 8

3.8 Row locking: select_for_update() . . . 8

3.9 WebSockets, Channels, consumers and groups . . . 9

3.10 A background worker . . . 9

3.11 Idempotent . . . 9

4. Map of the project: every folder and file 10

4.1 Architecture diagram . . . 11

5. The config folder 12

5.1 settings.py . . . 12

5.2 urls.py: the top-level address book . . . 12

5.3 asgi.py: the front door for Daphne . . . 12

5.4 mail.py, email_backends.py, context_processors.py . . . 13

6. The accounts app (people) 14

6.1 models.py: the Applicant . . . 14

6.2 forms.py: three forms . . . 14

6.3 views.py: register, register_done, activate . . . 15

6.4 services.py: activate_applicant() . . . 16

6.5 ratelimit.py: stopping guessers . . . 16

6.6 decorators.py: @applicant_required . . . 16

6.7 pdfs.py: the two PDF documents . . . 17

6.8 emails.py, admin.py, urls.py, templates . . . 17

7. The auctions app: data and admin 18

7.1 models.py: four tables . . . 18

7.2 Database rules (constraints) . . . 19
7.3 ER diagram (how the tables connect) . . . 20
7.4 importers.py: Excel import . . . 20
7.5 admin.py: the admin screens . . . 21
8. The auctions app: live bidding and closing **22**
8.1 views.py: three pages . . . 22
8.2 bidding.py: every bid rule, in one place . . . 22
8.3 consumers.py: AuctionConsumer (the WebSocket handler) . . . 23
8.4 realtime.py: everything sent to browsers . . . 24
8.5 routing.py . . . 24
8.6 auction_room.js: the browser side . . . 24
8.7 closing.py: closing auctions exactly once . . . 25
8.8 emails.py: winner email, sent once . . . 25
8.9 Management commands . . . 25
9. Complete stories: following the code **26**
9.1 Story: Priya registers . . . 26
9.2 Story: admin verifies, Priya activates and logs in . . . 26
9.3 Story: one bid, step by step . . . 26
9.4 Story: time runs out . . . 27
10. Security and privacy, summarised **28**
11. The automated tests **29**
11.1 The most important test: simultaneous bids . . . 29
12. Running the project **30**
12.1 Everyday commands . . . 30
12.2 Where to change things . . . 30
13. Questions your mentor might ask **31**
14. Glossary **32**

1. How to read this guide

This guide explains **every part** of the project, assuming you are new to Django. Read it in order the first time. Each chapter builds on the last one:

- **Chapters 2-3**: what the app does, and the basic ideas (Django, models, views, WebSockets...) explained with tiny examples.
- **Chapter 4**: a map of every folder and file.
- **Chapters 5-8**: each part of the code, file by file, with real excerpts.
- **Chapter 9**: complete stories ("what happens when someone registers / bids / wins") that follow the code from file to file.
- **Chapters 10-13**: security, tests, running the project, and questions your mentor might ask.
- **Chapter 14**: a glossary of every technical word used.

Beginner tip: Keep the project open in VS Code while you read. When the guide says `auctions/bidding.py`, open that file and look at the real code next to the explanation. The excerpts here are shortened; the real files have more comments.

2. The big picture: what the app does

The app runs **online auctions for vacant residential quarters** (company flats). An admin uploads the list of empty quarters, people register and pay a deposit, and then they bid against each other live in the browser. When time runs out, the highest bidder wins the quarter and gets an email.

2.1 The nine phases

#	Phase	What happens	Main files
1	Admin setup	Admin uploads an Excel file of quarters and creates auctions (start, end, minimum increment, anti-sniping extension).	auctions/importers.py, auctions/admin.py
2	Registration	Applicant fills a form: name, father's name, mobile, email, address, state, PIN. No password yet.	accounts/forms.py, accounts/views.py
3	Two PDFs emailed	System creates a 7-digit URN and an application number, builds an Application Form PDF (with QR code) and an Intimation Slip PDF, and emails both.	accounts/models.py, accounts/pdfs.py, accounts/emails.py
4	Pay EMD offline	Applicant pays the deposit (EMD) through SBI Collect using the URN. This happens outside the app. Admin ticks "verified".	accounts/admin.py
5	Activation	Applicant types email + URN + new password. A login account is created with username = URN.	accounts/forms.py, accounts/services.py
6	Lobby	Logged-in applicant sees live, upcoming and recently closed auctions.	auctions/views.py, lobby.html
7	Live bidding	Applicant joins an auction, gets a code like "Bidder 3", and bids. Every browser sees new bids instantly (WebSockets).	auctions/bidding.py, consumers.py, realtime.py, auction_room.js
8	Automatic closure	A background worker closes auctions whose time is up and picks the winner, exactly once.	auctions/closing.py, run_auction_worker.py
9	Winner email	Winner gets a congratulations email with quarter details and amount.	auctions/emails.py, winner_body.txt

2.2 The people who use it

Who	What they can do
Visitor (not logged in)	See home page, register, activate, log in, reset password.
Applicant (activated)	See the lobby, join auctions, bid. Only ever sees anonymous codes of other bidders.
Staff (admin)	Import quarters, create/cancel auctions, verify applicants, watch auctions (cannot bid).
Superuser	Everything staff can do, plus see the confidential list of who is "Bidder N" (Participation screen).

2.3 The technology, in one sentence each

Tool	What it is for
Python 3.13	The programming language everything is written in.
Django 5.2	A web framework: handles URLs, pages, forms, logins, database and admin for us.
PostgreSQL	The database. Needed (instead of SQLite) because it supports row locking, which keeps simultaneous bids safe.

Tool	What it is for
Django Channels	Adds WebSockets to Django, so the server can push new bids to browsers.
Daphne	The server program that runs Django + Channels (an ASGI server).
Redis (optional)	Lets several server processes share WebSocket messages. Not needed for one process.
openpyxl	Reads Excel (.xlsx) files.
ReportLab + qrcode	Create PDF files and QR code images.
django-environ	Reads secrets (passwords, keys) from a .env file instead of the code.
pypdf	Used only in tests, to read text back out of the PDFs we make.

3. Basic ideas you need first

If you already know a concept, skip its section. Each one has a tiny example.

3.1 What happens when you open a web page

Your browser sends a **request** ("GET /auctions/") to the server. The server runs some code and sends back a **response** (usually HTML). Then the connection is finished. This is **HTTP**. It is like posting a letter and getting one reply.

3.2 Django's pattern: Model - View - Template

Part	Job	Example in this project
Model	Describes a database table as a Python class.	class Quarter(models.Model) in auctions/models.py
View	A Python function that receives the request and returns a response.	def lobby(request) in auctions/views.py
Template	An HTML file with blanks that the view fills in.	auctions/templates/auctions/lobby.html
URL pattern	Connects a web address to a view.	path("", views.lobby) in auctions/urls.py

The three pieces working together (simplified)

```
# urls.py  - "when someone visits /auctions/, run views.lobby"
path("", views.lobby, name="lobby")

# views.py - get data, give it to a template
def lobby(request):
    auctions = Auction.objects.filter(status="OPEN")
    return render(request, "auctions/lobby.html", {"live": auctions})

# lobby.html - show it
{% for auction in live %}  <p>{{ auction.quarter.quarter_no }}</p>  {% endfor %}
```

3.3 The ORM: talking to the database in Python

ORM = Object-Relational Mapper. Instead of writing SQL, you write Python and Django turns it into SQL for you.

```
Quarter.objects.filter(status="VACANT")          # SELECT * FROM quarter WHERE status='VACANT'
Quarter.objects.get(quarter_no="B-214")          # exactly one row (error if none)
Quarter.objects.create(quarter_no="B-214", ...)  # INSERT
quarter.status = "ALLOTTED"; quarter.save()      # UPDATE
```

Migrations are files (in each `migrations/` folder) that record how the tables changed over time. `python manage.py makemigrations` writes them; `python manage.py migrate` applies them to the database.

3.4 Apps

A Django project is split into **apps**, each a folder for one topic. This project has two: **accounts** (people: applicants, registration, activation, login) and **auctions** (quarters, auctions, bids). **config** is not an app; it holds project-wide settings.

3.5 Forms and validation

A **Form** class lists the fields a user fills in and checks them. `form.is_valid()` runs all checks; methods named `clean_<field>()` tidy or check one field; `clean()` checks fields together. A **ModelForm** builds its fields from a model automatically.

3.6 Settings and the .env file

`config/settings.py` holds all configuration. Secrets (the SECRET_KEY, email password, database password) must never be written in the code, because code goes into git. So they live in a file called `.env` that is never committed. django-environ reads it:

```
# .env  (never committed)
SECRET_KEY=x8#k...long-random...
DATABASE_URL=postgres://sail_user:password@localhost:5432/sail_auction

# settings.py
SECRET_KEY = env('SECRET_KEY')
DATABASES = {'default': env.db('DATABASE_URL', default='sqlite:///db.sqlite3')}
```

3.7 Transactions: all or nothing

A **transaction** groups several database changes so that either **all** happen or **none** do. Example: when an auction closes we (1) mark the auction closed and (2) mark the quarter Allotted. If the program crashed between the two, we would have a closed auction with a quarter that still says "In Auction". Inside a transaction, a crash undoes both.

```
from django.db import transaction

with transaction.atomic():
    auction.status = "CLOSED"; auction.save()
    quarter.status = "ALLOTTED"; quarter.save()
# Leaving the "with" block = COMMIT (saved). An error inside = ROLLBACK (nothing saved).
```

3.8 Row locking: select_for_update()

Imagine two bidders both see "highest bid Rs. 4,500" and both send Rs. 4,600 at the same millisecond. Without care, both programs read 4,500, both think 4,600 is valid, and both save. That is a **race condition**.

`select_for_update()` tells PostgreSQL: "lock this row until my transaction ends". A second program that asks for the same lock must **wait**. So the bids are handled strictly one after the other: the second one then sees 4,600 and is rejected as too low.

```
with transaction.atomic():
    auction = Auction.objects.select_for_update().get(pk=7)   # others wait here
    if amount < auction.current_highest_bid + auction.min_increment:
        raise BidRejected(...)
    ... save the bid ...
# lock released at the end of the transaction
```

Watch out: SQLite ignores select_for_update(). That is why this project needs PostgreSQL for bidding. The concurrency tests are automatically skipped on SQLite.

3.9 WebSockets, Channels, consumers and groups

HTTP is "ask once, get one answer". For live bidding we need the **server to push** new bids to everyone without them asking. A **WebSocket** is a connection that stays open, like a phone call instead of a letter: both sides can talk at any time.

- **Django Channels** adds WebSocket support to Django.
- A **consumer** is to a WebSocket what a view is to an HTTP request. One consumer object exists for each open browser tab. Ours is `AuctionConsumer`.
- A **group** is a named list of connections. Everyone watching auction 7 is in group `auction_7`. Sending one message to the group delivers it to all of them.
- The **channel layer** is the post office that carries group messages. In development it is in memory; with several server processes it must be Redis.
- **ASGI** is the standard that lets a Python server handle both HTTP and WebSockets. **Daphne** is the ASGI server we use.

3.10 A background worker

Auctions must close when time runs out, even if nobody has a browser open. So a separate program runs all the time in its own terminal: `python manage.py run_auction_worker`. Every 5 seconds it looks for auctions past their deadline and closes them. We chose a simple management command instead of a bigger tool like Celery, so there are no extra services to install.

3.11 Idempotent

An action is **idempotent** if doing it twice has the same effect as doing it once. Pressing a lift button five times still calls the lift once. Our `close_auction()` is idempotent: the worker, a second worker and a browser may all try to close the same auction, but it is closed and awarded only once.

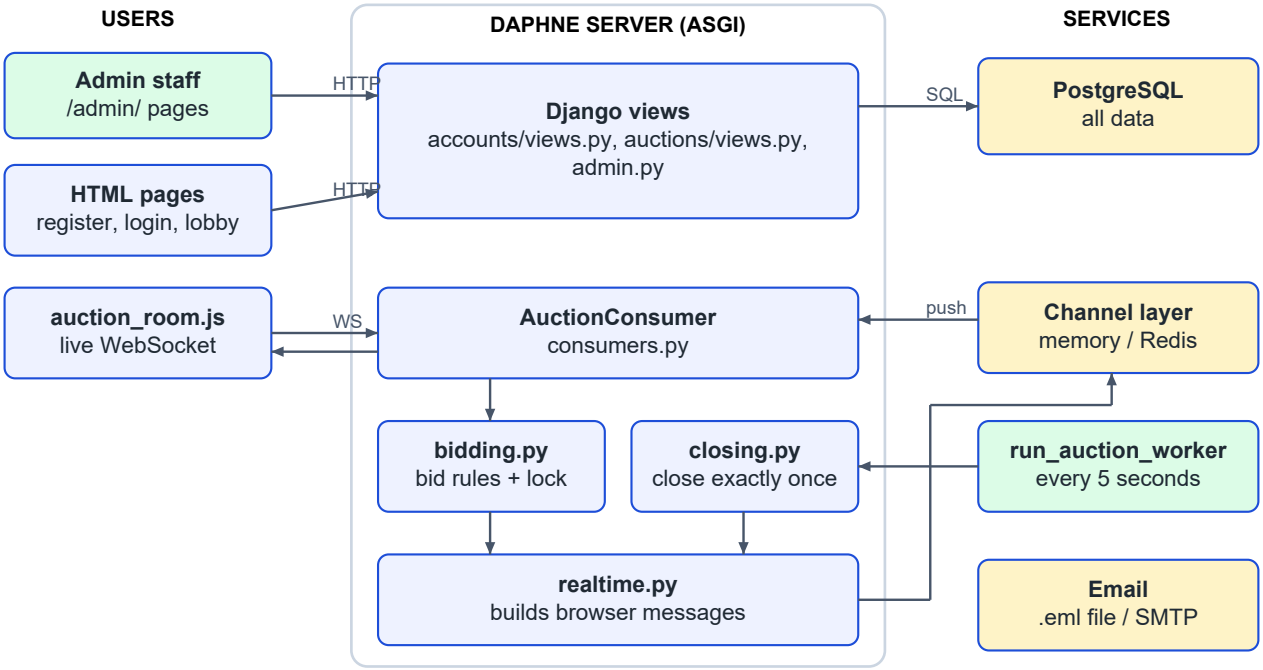
4. Map of the project: every folder and file

```
AUCTION/
├── manage.py               Django's command-line tool (runserver, migrate, test ...)
├── requirements.txt        Exact versions of every Python package
├── .env / .env.example     Secrets (not committed) / template showing what to put in .env
├── CLAUDE.md               Project brief and rules
├── PROGRESS.md             What is done, assumptions, next steps
├── README.md               Setup instructions
│
├── config/                 PROJECT-WIDE CONFIGURATION
│   ├── settings.py         All settings (reads .env)
│   ├── urls.py             Top-level URL map
│   ├── asgi.py             Entry point for Daphne: HTTP + WebSockets
│   ├── wsgi.py             Older HTTP-only entry point (unused by Daphne)
│   ├── mail.py             build_email(): one helper for every email
│   ├── email_backends.py   Development: saves emails as .eml files
│   └── context_processors.py  Values every template can use
│
├── accounts/               APP 1: PEOPLE
│   ├── models.py           Applicant model, URN generation
│   ├── forms.py            Registration, activation, login forms
│   ├── views.py            register, register_done, activate pages
│   ├── urls.py             /register/, /activate/, /login/, /password-reset/ ...
│   ├── services.py         activate_applicant(): creates the login account
│   ├── decorators.py       @applicant_required: who may open bidding pages
│   ├── ratelimit.py        Blocks password/URN guessing
│   ├── pdfs.py             Application Form + Intimation Slip PDFs
│   ├── emails.py           Sends the registration email with both PDFs
│   ├── admin.py            Applicant admin screen
│   ├── constants.py        List of Indian states
│   ├── templates/          HTML pages and email text
│   ├── migrations/         Database history
│   └── tests/              Automated tests
│
├── auctions/               APP 2: QUARTERS, AUCTIONS, BIDS
│   ├── models.py           Quarter, Auction, Participation, Bid
│   ├── importers.py        Excel import with full validation
│   ├── forms.py            Excel upload form
│   ├── admin.py            Admin screens + "Import from Excel" page
│   ├── views.py            lobby, auction_room, join
│   ├── urls.py             /auctions/, /auctions/7/, /auctions/7/join/
│   ├── bidding.py          ALL bid rules (with row locking)
│   ├── consumers.py        WebSocket handler (AuctionConsumer)
│   ├── routing.py          WebSocket URL: /ws/auctions/7/
│   ├── realtime.py         Builds every message sent to browsers
│   ├── closing.py          Closes auctions once, picks winner
│   ├── emails.py           Winner (and optional loser) emails
│   ├── management/commands/
│   │   ├── run_auction_worker.py   Background worker (loop)
│   │   ├── close_auctions.py       Close once and exit (for Task Scheduler)
│   │   └── seed_demo.py            Fake demo data
│   ├── static/auctions/    auction_room.js, countdown.js
│   ├── templates/          lobby, auction room, admin import page, email text
│   └── tests/              Automated tests
│
├── templates/              Shared templates: base.html, home.html, email footer
├── static/css/site.css     Shared styles
├── sample_data/            FAKE Excel files for import tests and the demo
├── scripts/                PowerShell helpers: run_server, run_worker, start_postgres ...
└── docs/                   ARCHITECTURE, DEMO_SCRIPT, DEPLOYMENT, HOW_TO_CHANGE
```

Beginner tip: A rule that helps you find things: **models.py** = what is stored, **views.py** = what a page does, **forms.py** = what a user types, **templates/** = what it looks like, **admin.py** = admin screens, and the extra files (bidding.py, closing.py ...) = business rules kept out of the views so they can be reused and tested.

4.1 Architecture diagram

How the pieces talk to each other while the app runs:



Every part inside the server reads and writes PostgreSQL. Emails are sent by accounts/emails.py (registration) and auctions/emails.py (winner), called from the views and from closing.py.

- Browsers talk to **Daphne** over HTTP (normal pages) and WebSocket (live auction page).
- Bids go WebSocket -> **AuctionConsumer** -> **bidding.py** -> PostgreSQL.
- After a bid is saved, **realtime.py** builds a message and the **channel layer** delivers it to every tab in that auction's group.
- The **worker** calls **closing.py** every few seconds; closing then sends the winner email.

5. The config folder

5.1 settings.py

Read from top to bottom, it sets up:

Section	What it does
Secrets & security	SECRET_KEY, DEBUG and ALLOWED_HOSTS come from .env. When DEBUG is False (production) it switches on HTTPS-only cookies, HTTPS redirect and HSTS automatically.
INSTALLED_APPS	'daphne' first (so runserver also serves WebSockets), Django's built-in apps, 'channels', and our two apps.
MIDDLEWARE	Code that runs on every request: security headers, sessions, CSRF protection, login, messages.
CHANNEL_LAYERS	If REDIS_URL is set use Redis, otherwise in-memory.
DATABASES	From DATABASE_URL; falls back to SQLite if missing.
TIME_ZONE	'Asia/Kolkata' with USE_TZ=True: times are stored in UTC and shown in IST.
Email	If EMAIL_HOST_PASSWORD is set, send real email via SMTP; otherwise save each email as a .eml file in sent_emails/.
Login	LOGIN_URL, where to go after login (lobby) and logout (home). Reset links expire in 1 hour.
PROJECT SETTINGS	Business values at the bottom: EMD_AMOUNT, anti-snipe window (60 s), bid flood limit (1 s), one-quarter-at-a-time rule, worker interval (5 s), notify losers (off)...

Every business value can be changed in .env without touching code

```
AUCTION_ANTI_SNIPE_WINDOW_SECONDS = env.int('AUCTION_ANTI_SNIPE_WINDOW_SECONDS', default=60)
AUCTION_ONE_QUARTER_AT_A_TIME     = env.bool('AUCTION_ONE_QUARTER_AT_A_TIME', default=True)
AUCTION_NOTIFY_LOSERS             = env.bool('AUCTION_NOTIFY_LOSERS', default=False)
```

5.2 urls.py: the top-level address book

```
urlpatterns = [
    path('', TemplateView.as_view(template_name='home.html'), name='home'),
    path('', include('accounts.urls')),        # /register/, /activate/, /login/ ...
    path('auctions/', include('auctions.urls')),  # /auctions/, /auctions/7/ ...
    path('admin/', admin.site.urls),
]
```

`include()` hands everything under a prefix to an app's own urls.py. Each URL has a **name**, so templates write `{% url 'auctions:lobby' %}` instead of hard-coding the address.

5.3 asgi.py: the front door for Daphne

```
application = ProtocolTypeRouter({
    "http": django_asgi_app,                       # normal pages -> Django
    "websocket": AllowedHostsOriginValidator(       # reject sockets opened by other websites
        AuthMiddlewareStack(                        # read the login cookie -> scope["user"]
            URLRouter(websocket_urlpatterns))),     # /ws/auctions/7/ -> AuctionConsumer
})
```

Think of it as a receptionist: HTTP visitors go to Django; WebSocket visitors are checked (right website? who is logged in?) and then sent to the consumer.

5.4 mail.py, email_backends.py, context_processors.py

- **mail.py**: `build_email(template, context, to)` loads `<template>_subject.txt` and `_body.txt`, fills them in and returns an email ready to send. So email **wording** lives in .txt files, not in Python.
- **email_backends.py**: in development, saves each email as a `.eml` file you can double-click to open in Outlook/Mail, attachments included.
- **context_processors.py**: makes AUCTION_TITLE and RESET_LINK_HOURS available in every template automatically.

6. The accounts app (people)

6.1 models.py: the Applicant

One row per registered person. Fields are grouped in three parts:

Group	Fields	Notes
Entered at registration	name, father_name, mobile, email, address, state, pin_code	Validators check mobile = 10 digits starting 6-9, PIN = 6 digits. Email is unique (case-insensitive).
Made by the system	application_no, application_date, urn, urn_date, documents_emailed_at	application_no looks like QA2026000001. URN is a random 7-digit number.
Status	is_verified, is_activated, user	is_verified is ticked by admin after checking the EMD payment. user links to Django's User, created only at activation.

How the URN is generated

```
def generate_urn():
    """Random 7-digit URN, 1000000-9999999. Not guessable."""
    return str(1_000_000 + secrets.randbelow(9_000_000))
```

`secrets` is Python's module for **cryptographically secure** random numbers (unlike `random`, which is predictable). This matters because the URN is half of what you need to activate an account.

save(): what happens when an Applicant is saved

- Email is lower-cased so "A@x.com" and "a@x.com" are the same.
- If there is no URN, `_insert_with_new_urn()` tries a random one. If the database says it is taken (IntegrityError from the unique constraint), it tries again, up to 10 times. The **database** is the referee, so two simultaneous registrations can never get the same URN.
- After the row exists (and so has an id), the application number is built from the year and the id: `f"QA{year}{id:06d}"` -> QA2026000001.
- All of this runs inside one transaction, so nobody ever sees a half-made applicant.

Constraints and the for_user() helper

- `applicant_activated_has_user`: the database refuses an activated applicant with no login account.
- `applicant_email_unique_ci`: unique email, ignoring upper/lower case.
- `Applicant.for_user(user)` returns the activated applicant behind a login, or None. It is the **one** place this lookup is written, and views, the consumer and decorators all use it.
- `on_delete=models.PROTECT` on `user`: deleting a User that has an applicant is refused, so audit history is never silently lost.

6.2 forms.py: three forms

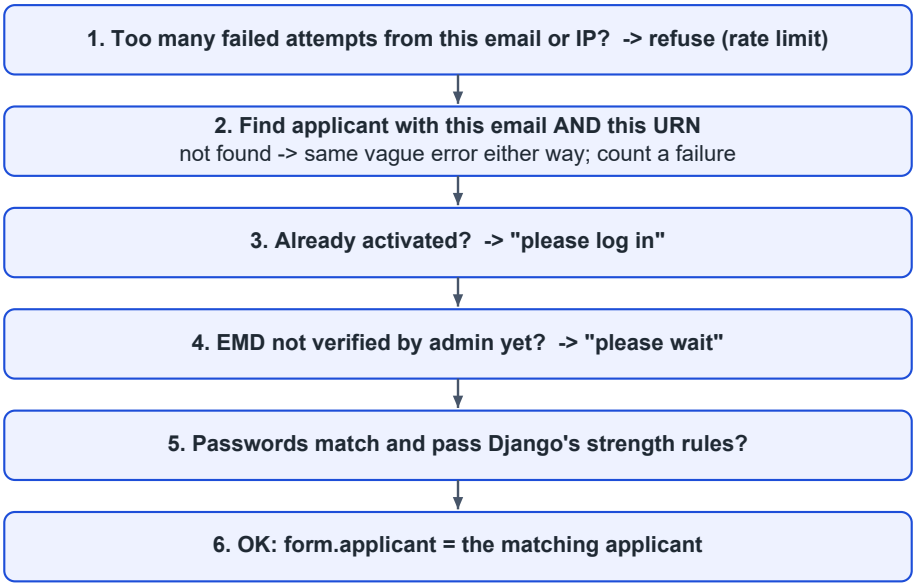
RegistrationForm (a ModelForm)

Builds its fields from the Applicant model, adds a required **declaration** checkbox, and tidies input before validation:

```
def clean_mobile(self):
    # Accept "+91 98765-43210", "098765 43210", etc. -> "9876543210"
    digits = re.sub(r"[\s\-()]", "", self.cleaned_data["mobile"])
    if digits.startswith("+91"):
        digits = digits[3:]
    elif len(digits) == 11 and digits.startswith("0"):
        digits = digits[1:]
    return digits
```

ActivationForm

Asks for email, URN, password and password again. Its `clean()` method is the security heart of activation:



Notice step 2 gives the **same message** whether the email or the URN is wrong. If it said "URN wrong", an attacker would learn the email is registered. Only after both are correct does it explain the exact problem.

UrnLoginForm

Django's normal login form, with the username field labelled "URN" and the same rate limiting added around it.

6.3 views.py: register, register_done, activate

```
def register(request):
    if request.method == "POST":
        form = RegistrationForm(request.POST)
        if form.is_valid():
            with transaction.atomic():
                applicant = form.save()          # URN + application no. created here
            try:
                send_registration_documents(applicant)   # PDFs + email
                email_ok = True
            except Exception:
                logger.exception(...)
                email_ok = False
            request.session["registered_application_no"] = applicant.application_no
            return redirect("accounts:register_done")   # Post/Redirect/Get
    else:
        form = RegistrationForm()
    return render(request, "accounts/register.html", {"form": form})
```

- **GET** (just opening the page): show an empty form. **POST** (submitting): validate and save.
- The email is sent **after** saving. If the mail server is down, the registration is not lost; the admin can resend documents later.
- **Post/Redirect/Get**: after a successful POST we redirect to a success page. If the user presses refresh, the browser reloads the success page instead of submitting the form twice.
- `activate`: if the form is valid it calls `activate_applicant()`, logs the user in, and sends them to the lobby.

6.4 services.py: activate_applicant()

```
def activate_applicant(applicant, password):
    with transaction.atomic():
        applicant = Applicant.objects.select_for_update().get(pk=applicant.pk)
        if applicant.is_activated:
            raise ActivationError("This account is already activated.")
        if not applicant.is_verified:
            raise ActivationError("EMD payment not verified yet.")
        user = User.objects.create_user(username=applicant.urn,
                                        email=applicant.email, password=password)
        applicant.user = user
        applicant.is_activated = True
        applicant.save(update_fields=["user", "is_activated", "updated_at"])
    return user
```

- The applicant row is **locked**, so two tabs activating at once cannot create two accounts.
- `create_user()` **hashes** the password (stores a one-way scrambled version). Nobody, not even the admin, can read the real password.
- Username = URN. The password is stored in exactly one place: Django's User table.
- Why a separate "services" file? So the same rule runs whether activation comes from the web page, a test, or the seed_demo command.

6.5 ratelimit.py: stopping guessers

Counts failed attempts in Django's cache, with two counters per attempt:

Counter	Default limit	Why
Per email / URN	5 failures in 15 minutes	Stops someone guessing the URN of one account.
Per IP address	50 failures in 15 minutes	Higher, because a whole office may share one IP, but still stops one machine trying many accounts.

The email/URN is stored as a SHA-256 hash in the cache key, so personal data is not kept in the cache. A success clears the account's counter.

6.6 decorators.py: @applicant_required

A **decorator** wraps a function to add behaviour before it runs. `@applicant_required` on a view means: not logged in -> go to login page; logged in but not an activated applicant and not staff -> "403 Forbidden"; otherwise run the view.

```
@applicant_required
def lobby(request):
    ...
```

6.7 pdfs.py: the two PDF documents

Uses ReportLab's **platypus**: you build a list ("story") of paragraphs, tables and images and ReportLab lays them out on A4 pages. Both functions return the PDF as **bytes** in memory (no file saved), ready to attach to an email.

PDF	Contents
Application Form	Training notice, title, application number and date, a QR code, auction details, applicant details, address, declaration, signature line.
Intimation Slip	URN and URN date, application number, name, mobile, EMD amount, SBI Collect payment steps (placeholders), and a warning to keep the URN secret.

- The QR code contains only `QA-APPLICATION|<application no>|<date>`. Deliberately **not** the URN, name or phone, because anyone can scan a printed QR.
- `_esc()` escapes < > & in user-typed text, because ReportLab treats <...> as formatting tags. Without it, a name like "<b>" would break the PDF.

6.8 emails.py, admin.py, urls.py, templates

- **emails.py**: `send_registration_documents(applicant)` builds the email from the .txt templates, attaches both PDFs, sends it, and records `documents_emailed_at`.
- **admin.py**: Applicant list with search and filters. System fields (application_no, is_activated, user...) are read-only. An action "Resend registration documents" re-sends the PDFs. Admin ticks **verified** here after checking the EMD payment.
- **urls.py**: register, activate, login and logout, and the four password-reset pages, which use Django's built-in, token-based reset views (the link in the email contains a signed token that expires after 1 hour).
- **templates/accounts/**: the HTML for each page; **emails/** holds the email wording.

7. The auctions app: data and admin

7.1 models.py: four tables

Quarter

Field	Type	Notes
quarter_no	Text, unique	e.g. "B-214". Always saved in UPPER CASE; uniqueness ignores case.
sector, location_block, floor	Text	Floor is text: "Ground", "1st" ...
quarter_type	A / B / C / D	Shown as "Type B".
area_sqft, base_rate	Decimal	Must be > 0 (database check constraints). Decimal, never float, for money.
status	VACANT / IN_AUCTION / ALLOTTED	Changes automatically as auctions open and close.

Beginner tip: Why **DecimalField** for money? Floats are binary approximations: in Python 0.1 + 0.2 gives 0.30000000000000004. Decimal stores exact rupees and paise.

Auction

Field	Meaning
quarter	Which quarter is auctioned (PROTECT: a quarter with auction history can't be deleted).
start_time, end_time	end_time is the *current* deadline; anti-sniping can push it later.
original_end_time	The deadline before any extensions (for audit).
status	OPEN, CLOSED (won), CLOSED_NO_BIDS, CANCELLED.
min_increment	Each new bid must beat the highest by at least this (default Rs. 100, placeholder).
anti_snipe_extension	How much a last-minute bid extends the deadline (default 2 minutes, placeholder).
current_highest_bid	Kept up to date by bidding.py.
winning_bid, closed_at	Final result, written once by closing.py.
winner_notified_at, losers_notified_at	When result emails were sent: prevents sending twice.

Status vs phase. The database stores only what the clock cannot tell us (open, closed, cancelled). Whether an OPEN auction is *Upcoming*, *Live* or *Closing* is **calculated** from the time every time it is needed:

```
def phase(self, now=None):
    now = now or timezone.now()
    if self.status == self.Status.CANCELLED: return self.Phase.CANCELLED
    if self.status != self.Status.OPEN:      return self.Phase.CLOSED
    if now < self.start_time:                return self.Phase.UPCOMING
    if now < self.end_time:                  return self.Phase.LIVE
    return self.Phase.CLOSING     # deadline passed, closure job hasn't run yet
```

So no background job is needed to switch an auction from "Upcoming" to "Live"; the page is always right. Other useful methods:

- `minimum_next_bid`: base rate if there are no bids yet, otherwise highest + min_increment.

- `save()`: when a new OPEN auction is created, the quarter becomes IN_AUCTION (same transaction).
- `cancel()`: locks the auction, sets CANCELLED, makes the quarter VACANT again, keeps the bids for audit, and tells open browsers.
- `clean()`: used by the admin form: can't auction an allotted quarter; a new auction must end in the future.

Participation: the anonymous code

One row per (applicant, auction). `bidder_number` is 1, 2, 3 ... in joining order within each auction, shown as "Bidder 3". The same person can be Bidder 1 in one auction and Bidder 5 in another, so codes can't be linked across auctions.

> **Privacy:** This table is the **only** link between a real person and their code. Public pages, bid history and WebSocket messages show codes only. Its admin screen is visible to superusers only. The identity is kept (not deleted) so the winner can be notified and disputes resolved; do not claim it is cryptographically anonymous.

Bid

One row per **accepted** bid. Rejected attempts are never stored. Bids are never edited or deleted: they are the audit trail. Besides auction, participation, amount and time, each bid records:

- `client_bid_id`: a random id (UUID) made by the browser for each click. If the same id arrives twice (double-click, resend after reconnect), it is recognised as the same bid.
- `previous_highest` and `deadline_extended_to`: what the highest bid was before, and whether this bid extended the deadline (audit).

7.2 Database rules (constraints)

Rules enforced by **PostgreSQL itself**, so even buggy Python code can't break them:

Constraint	Rule
quarter_no_unique_ci	Quarter numbers are unique, ignoring case.
*_positive checks	Area, base rate, min increment, bid amount must be > 0.
auction_end_after_start	An auction ends after it starts.
one_open_auction_per_quarter	A quarter can have only one OPEN auction at a time (partial unique index).
quarter_won_only_once	A quarter can be WON only once, ever. The last line of defence against double awarding.
winning_bid_matches_status	"Closed - won" always has a winning bid; other statuses never do.
one_participation_per_applicant_per_auction	One code per person per auction.
unique_bidder_number_per_auction	No two people share a code in the same auction.
no_duplicate_bid_submission	The same browser click (client_bid_id) can't create two bids.
unique_bid_amount_per_auction	Two bids in one auction never have the same amount.
applicant_activated_has_user	An activated applicant always has a login.

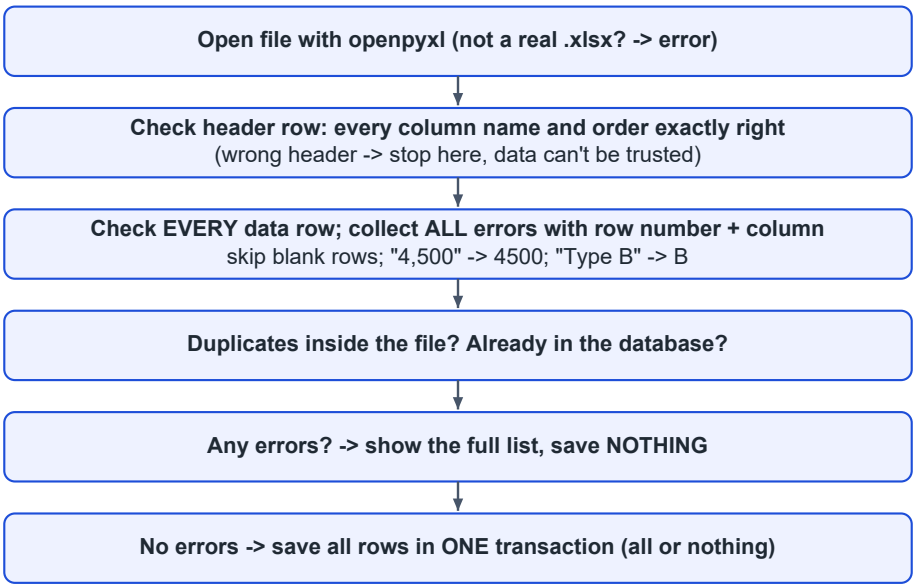
7.3 ER diagram (how the tables connect)

```
User 1 ─── 0..1 Applicant              (login created at activation)
                  │ 1
                  │
                  ▼ many
Quarter 1 ── many Auction 1 ── many Participation  ("Bidder N")
                    │ 1                 │ 1
                    │                   │
                    ▼ many              ▼ many
                   Bid ◄────────────────┘
                    ▲
Auction.winning_bid ┘  (0..1, set when closed)
```

7.4 importers.py: Excel import

The admin uploads an .xlsx file with exactly these columns:

```
Quarter No | Sector | Location / Block | Type | Floor | Area (sq ft) | Base Rate (Rs.) | Status
```



Example error list for `sample_data/quarters_invalid.xlsx`:

```
On an empty database. If B-214 etc. already exist, "already exists in the database" errors are added
Row 3, Quarter No: 'B-214' is duplicated in this file (first seen at row 2).
Row 4, Area (sq ft): must be a number (got 'abc')
Row 5, Base Rate (Rs.): must be greater than 0 (got -2400)
Row 6, Quarter No: is required
```

And for `quarters_bad_header.xlsx` (first column named "Quarter Number"):

```
Row 1, Column A: expected header 'Quarter No', found 'Quarter Number'
```

Other details: max 2000 rows per file, max 5 MB upload (forms.py), Status must be "Vacant", and if another admin saves the same quarter during the import, the database constraint catches it and nothing is imported. The result is returned as an `ImportResult` (count created + list of `RowError`).

7.5 admin.py: the admin screens

Screen	Special behaviour
Quarters	Extra "Import from Excel" button and page (custom admin URL).
Auctions	New auctions can only pick Vacant quarters. Once bidding starts, start time, increment and extension are locked (fairness). Closed auctions are fully read-only. No delete button (audit) - use the **Cancel** action. Actions: cancel, close if due, send result emails. Bids shown inline with codes only. Editing an open auction pushes the change to browsers.
Bids	Read-only audit list: no add, edit or delete. Shows codes, not names.
Participations	The confidential who-is-who list. Superusers only, read-only.

8. The auctions app: live bidding and closing

8.1 views.py: three pages

View	URL	What it does
lobby	/auctions/	Lists OPEN auctions split into Live and Upcoming, plus recently closed ones (last 7 days). Marks which ones you joined. Shows quarter info only, never bidder info.
auction_room	/auctions/7/	The bidding page. Sends the page with a first snapshot of the auction so it looks right even before the WebSocket connects. Shows Join button, bid form, or "watching" for staff.
join	/auctions/7/join/ (POST)	Calls join_auction() to get a Bidder N code, then redirects back to the room.

8.2 bidding.py: every bid rule, in one place

join_auction(auction, applicant)

Returns the existing participation if you already joined. Otherwise checks the auction is upcoming or live and you are verified + activated, then **locks the auction row** and gives you number = highest existing number + 1. The lock means two people joining at the same moment can't both get "Bidder 4".

place_bid(auction_id, applicant, amount, client_bid_id)

First the input is cleaned: `parse_amount()` accepts "4,600", rejects text, negatives, more than 2 decimal places, and absurdly large numbers. Then, inside a transaction holding the lock on the auction row, the seven rules run in order:

Rule	Check	If it fails
1	Auction is LIVE (started, deadline not passed, not closed).	"Bidding has not started" / "auction has closed"
2	You joined this auction.	"Join the auction before bidding"
3	Same client_bid_id already accepted?	Not an error: returns the earlier bid marked duplicate
7	One quarter at a time: you have no bids in another auction that is still open (also locks the applicant row).	"You are already bidding for quarter X"
4	Amount >= base rate (first bid) or >= highest + min increment.	"Your bid must be at least Rs. ..."
5	You don't already hold the highest bid (assumption).	"You already hold the highest bid"
6	If less than 60 s remain, push deadline to now + extension (anti-sniping).	(not a rejection)

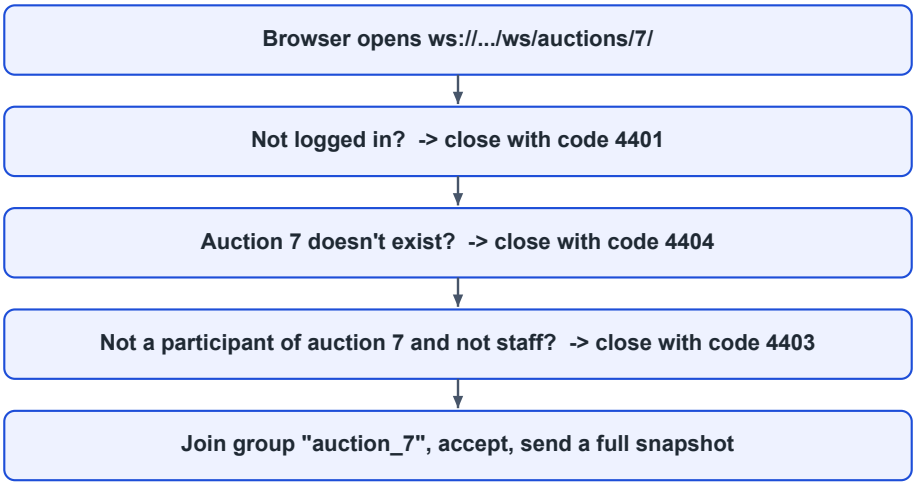
If all pass: save the Bid, update `current_highest_bid` (and `end_time` if extended), commit, and **only after commit** ask realtime.py to broadcast it.

```
with transaction.atomic():
    # THE LOCK: every other bid for this auction waits here until we commit.
    auction = (Auction.objects.select_for_update(of=("self",))
               .select_related("quarter").get(pk=auction_id))
    ... rules 1-7 ...
    bid = Bid.objects.create(...)
    auction.current_highest_bid = amount
    auction.save(update_fields=[...])
    transaction.on_commit(lambda: realtime.broadcast_bid(bid.pk))
```

- **Why lock the auction row and not a whole table?** Bids in one auction wait for each other, but different auctions never block each other, so many auctions can run at once.
- **Why broadcast after commit?** If something failed and the transaction rolled back, nobody should have seen that bid on screen.
- **Lock order is always auction then applicant.** Always taking locks in the same order prevents a *deadlock* (two programs each waiting forever for the other's lock).
- **Anti-sniping**: "sniping" = bidding in the last second so nobody can respond. Extending the deadline gives others a fair chance to reply.

8.3 consumers.py: AuctionConsumer (the WebSocket handler)

connect(): who may open the socket



This is **per-auction authorisation**: joining auction 7 does not let you connect to auction 8.

receive_json(): messages from the browser

Browser sends	Consumer does
{"type": "bid", "amount": "4600", "client_bid_id": "..."}	handle_bid(): check you are a participant, refuse if you bid less than 1 second ago (anti-flooding), call place_bid(), reply bid_result ok / not ok to **this tab only**.
{"type": "sync"}	Send a fresh snapshot. If the deadline has passed, close the auction first (safe because closing is idempotent).

auction_message(): messages from the group

When realtime.py sends to group `auction_7` with `"type": "auction.message"`, Channels calls `auction_message()` on every consumer in that group, and each one forwards the message to its browser.

Beginner tip: This is a **synchronous** consumer (JsonWebsocketConsumer). Channels runs it in a worker thread, so it can use the normal Django ORM just like a view. `async_to_sync()` is used to call the channel layer's async functions from this normal code.

8.4 realtime.py: everything sent to browsers

All messages are built in this one file so it is easy to check that **no private data** leaks. A message contains only codes, amounts and times:

Example bid_accepted message: no name, email, URN or mobile anywhere

```
{
  "type": "bid_accepted",
  "bid": {"bidder": "Bidder 3", "amount": "4600.00", "time": "2026-10-09T20:15:03+05:30"},
  "extended": false,
  "phase": "LIVE", "highest_bid": "4600.00", "minimum_next_bid": "4700.00",
  "end_time": "2026-10-09T21:00:00+05:30", "server_now": "2026-10-09T20:15:03+05:30", ...
}
```

Function	Purpose
group_name(id)	"auction_7" for auction 7.
auction_state()	The live numbers: phase, highest, minimum next, end time, server time, result.
snapshot()	Full picture on (re)connect: state + your own code + the last 20 bids.
broadcast_bid()	Sends bid_accepted to the whole group.
broadcast_state()	Sends state to the group when the auction changes without a bid (closed, cancelled, admin edited the time).

`server_now` lets the browser correct its countdown if the user's computer clock is wrong.

8.5 routing.py

```
websocket_urlpatterns = [path("ws/auctions/<int:auction_id>/", AuctionConsumer.as_asgi())]
```

The WebSocket version of urls.py.

8.6 auction_room.js: the browser side

- Opens the WebSocket and keeps it open. Draws every snapshot, bid_accepted and state message.
- When you click **Place bid** it creates a random `client_bid_id`, sends the bid, and disables the button until the server answers (no double bids). If no answer comes in 10 s, it unlocks and asks for a resync.
- **Reconnects automatically** after a network drop, waiting 1 s, 2 s, 4 s ... up to 15 s. The server sends a fresh snapshot on every connect, so the page is always correct after reconnecting.
- When the tab becomes visible again (laptop woke up), it asks for a sync.
- Countdown: when its clock reaches zero it asks the server (at most every 3 s) instead of deciding itself. **The server is always the authority.**
- Shows "You have been outbid by Bidder 5" and keeps the bid box at the new minimum.
- Uses `textContent`, never `innerHTML`, for data, so nothing from a message can run as a script (XSS protection).

countdown.js is a smaller script for the lobby: it shows "Ends in 00:12:30" and reloads the page once when a countdown hits zero, so the auction moves to its next section.

8.7 closing.py: closing auctions exactly once

```
def close_auction(auction_id, now=None):
    with transaction.atomic():
        auction = Auction.objects.select_for_update(of=("self",))...get(pk=auction_id)
        if auction.status != Auction.Status.OPEN or now < auction.end_time:
            return ClosureResult(..., closed_now=False)     # nothing to do
        winning_bid = auction.bids.order_by("-amount", "created_at").first()
        if winning_bid:
            auction.status = CLOSED;          auction.winning_bid = winning_bid
            new_quarter_status = ALLOTTED
        else:
            auction.status = CLOSED_NO_BIDS;  new_quarter_status = VACANT
        auction.closed_at = now
        auction.save(...)
        Quarter.objects.filter(pk=auction.quarter_id).update(status=new_quarter_status)
        transaction.on_commit(lambda: _after_close(auction.pk))   # broadcast + email
```

Why this can never award a quarter twice:

- **Lock**: a second closer waits at select_for_update().
- **Re-check**: when it gets the lock, the status is no longer OPEN, so it does nothing.
- **Database backstop**: the `quarter_won_only_once` constraint refuses a second CLOSED auction for the same quarter.
- Highest amount wins; if amounts were ever equal, the earliest bid wins (placeholder rule).
- No bids -> CLOSED_NO_BIDS and the quarter goes back to Vacant so it can be auctioned again.

Other functions: `close_due_auctions()` closes every OPEN auction past its deadline; `retry_unsent_notifications()` retries emails that failed earlier (for example, SMTP was down).

8.8 emails.py: winner email, sent once

`notify_winner()` locks the auction, checks `winner_notified_at` is empty, sends the email, and saves the time, all in one transaction. If sending fails, the transaction rolls back, the time stays empty, and the worker tries again next round. Loser emails work the same way but are **off by default**. The wording is in `winner_body.txt` / `loser_body.txt`.

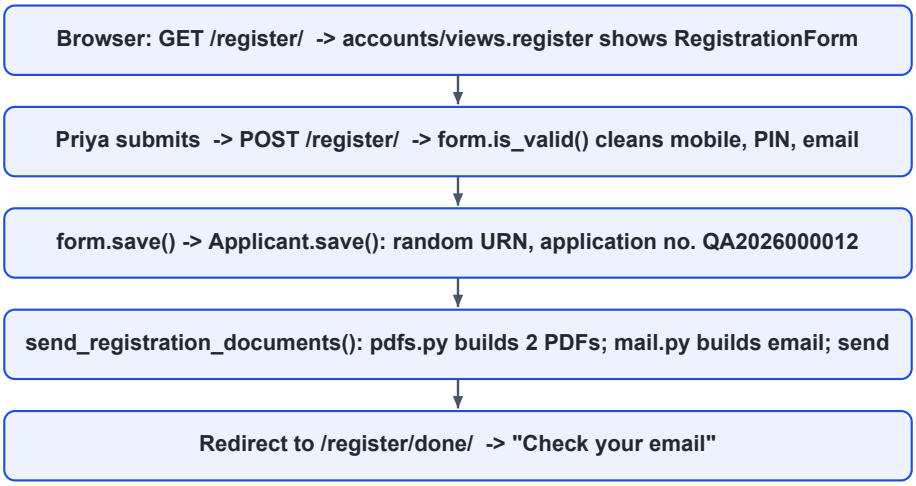
Note: Honest limit (also written in the code): if the email reaches the mail server but the database commit then fails, a retry could send it twice. Exactly-once delivery to an outside mail server is not possible; the window is tiny.

8.9 Management commands

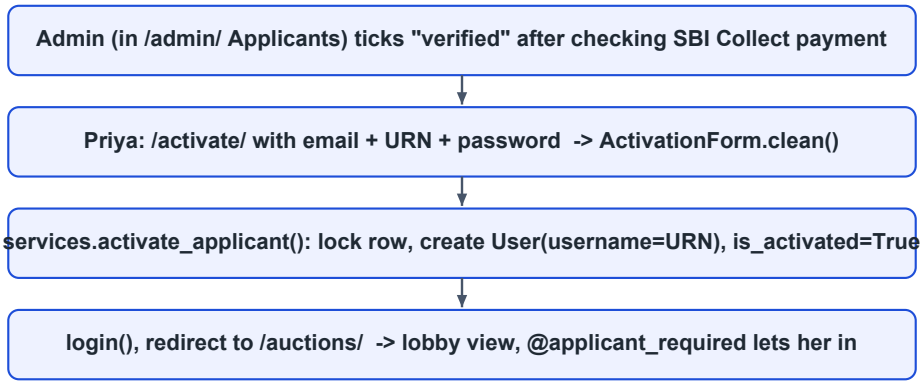
Command	What it does
run_auction_worker	Loops forever: every 5 s close due auctions and retry unsent emails. One failing round is logged and does not stop the worker. --rounds 1 runs once (tests).
close_auctions	Does one round and exits. For Windows Task Scheduler / cron.
seed_demo	Creates FAKE demo data: 8 quarters from the sample Excel, 2 live + 2 upcoming auctions, 3 activated applicants (password Demo-Pass-2026) and 1 waiting for activation. Refuses to run when DEBUG is False.

9. Complete stories: following the code

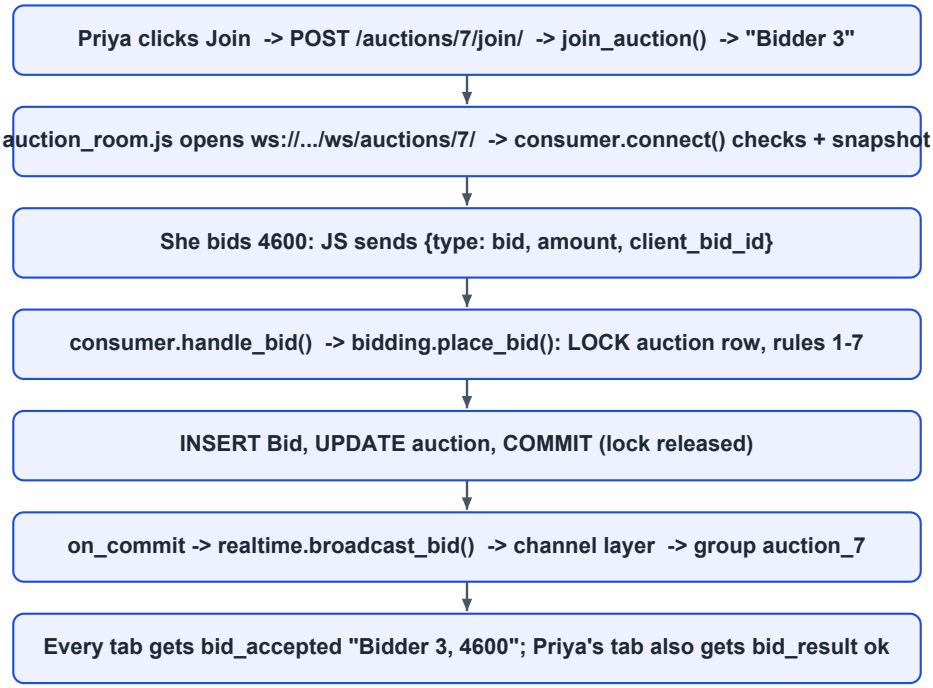
9.1 Story: Priya registers



9.2 Story: admin verifies, Priya activates and logs in

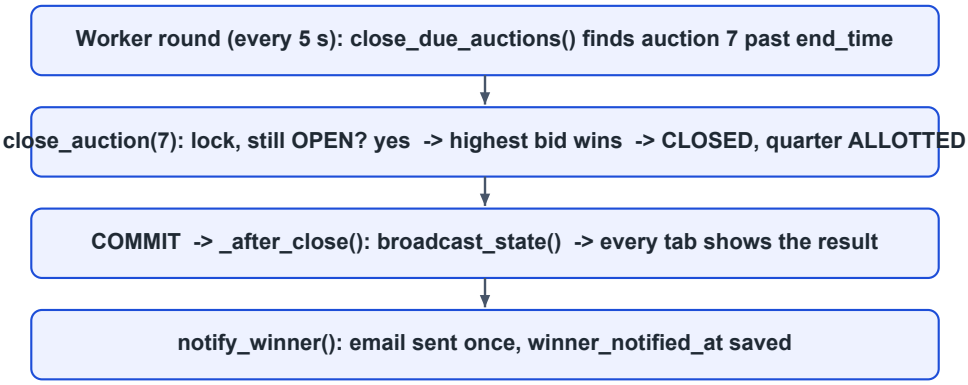


9.3 Story: one bid, step by step



If two bidders send 4,600 at the same instant, the second one waits at the lock, then sees the highest is now 4,600 and gets "Your bid must be at least Rs. 4,700.00".

9.4 Story: time runs out



At the same moment, a browser whose countdown hit zero may send "sync", and the consumer also calls close_auction(7). One of them gets the lock first and closes it; the other waits, re-checks, sees CLOSED, and does nothing.

10. Security and privacy, summarised

Threat	Protection	Where
Secrets leaked via git	.env file, never committed; no defaults for SECRET_KEY	settings.py, .gitignore
Guessing someone's URN to activate their account	Needs email AND URN; same vague error; rate limit 5 per account / 50 per IP; random secure URNs	forms.py, ratelimit.py, models.py
Stolen passwords	Django password hashing; strength validators; reset via expiring token link	services.py, settings.py
Two activations at once	Row lock on applicant	services.py
Fake bids / tampered browser	Every rule checked on the server; browser decides nothing	bidding.py
Simultaneous bids	select_for_update on auction row + unique amount constraint	bidding.py, models.py
Double-click / resend	client_bid_id + unique constraint; button disabled while pending	bidding.py, auction_room.js
Bid flooding	1 bid per second per connection	consumers.py
Watching an auction you didn't join	Per-auction authorisation on connect (4403)	consumers.py
Another website opening our WebSocket	AllowedHostsOriginValidator	asgi.py
Form forgery from other websites (CSRF)	Django CSRF middleware and {% csrf_token %}	settings.py, templates
Identity leaking	Codes only in pages, messages, bid admin; Participation admin superuser-only; QR has no personal data; __str__ shows application no., not names	realtime.py, admin.py, pdfs.py
Script injection (XSS)	Django auto-escaping; JS uses textContent; PDF text escaped	templates, auction_room.js, pdfs.py
Quarter awarded twice	Lock + re-check + quarter_won_only_once constraint	closing.py, models.py
Duplicate emails	Sent-at timestamps checked under lock	emails.py
Insecure production	DEBUG=False turns on HTTPS redirect, secure cookies, HSTS	settings.py

11. The automated tests

Tests are Python code that uses the app and checks the result automatically. There are **196 test functions** across 14 test files. Run them all with:

```
python manage.py test
```

File	Tests	What it checks
accounts/tests/test_models.py	10	URN format and uniqueness, application number, email case, constraints.
accounts/tests/test_registration.py	9	Form cleaning, validation errors, success page, email sent.
accounts/tests/test_generation.py	8	URN retry logic when a URN is already taken.
accounts/tests/test_documents.py	6	PDF contents (read back with pypdf); QR has no private data.
accounts/tests/test_activation.py	12	Wrong email/URN, not verified, already active, rate limit, success.
accounts/tests/test_login_and_reset.py	10	Login with URN, lock-out, password reset email and link.
auctions/tests/test_import.py	20	Valid, invalid and bad-header sample files; all errors reported; atomic.
auctions/tests/test_models.py	18	Phases, constraints, cancel, quarter status changes.
auctions/tests/test_admin.py	6	Admin import page, permissions, read-only rules.
auctions/tests/test_lobby.py	14	Who can see the lobby, sections, no identity leaks.
auctions/tests/test_auction_room.py	13	Room page, joining, codes, staff watching.
auctions/tests/test_bidding.py	28	Every bid rule: too low, not live, duplicate, anti-sniping, one-quarter rule.
auctions/tests/test_consumers.py	13	WebSocket connect permissions, bidding over socket, privacy of messages.
auctions/tests/test_closing.py	19	Winner chosen, no-bids outcome, idempotency, emails sent once, retries.
auctions/tests/test_concurrency.py	8	Real simultaneous bids from 10 threads on PostgreSQL.
auctions/tests/test_seed_demo.py	2	Demo data command works and refuses on production.

11.1 The most important test: simultaneous bids

Ten threads, each a different bidder with its own database connection, wait at a **barrier** (a starting line) and are released at the same instant, all bidding Rs. 4,500:

```
results = run_simultaneously([self.bid_job(a, "4500") for a in self.bidders])
self.assertEqual(len(accepted), 1)               # exactly one accepted
self.assertEqual(len(rejected), THREADS - 1)     # nine rejected
self.assertTrue(all(r.code == "too_low" for r in rejected))
self.assertEqual(Bid.objects.count(), 1)
```

If you remove `select_for_update` from bidding.py, this test fails. That is the proof the lock is doing its job. It uses **TransactionTestCase** because the normal TestCase wraps each test in a transaction that other connections cannot see.

12. Running the project

12.1 Everyday commands

Goal	Command
Start PostgreSQL + website	.\scripts\run_server.ps1 (then open http://127.0.0.1:8000)
Start the closing worker (second terminal)	.\scripts\run_worker.ps1
Create the tables	venv\Scripts\python.exe manage.py migrate
Make an admin login	venv\Scripts\python.exe manage.py createsuperuser
Fake demo data	venv\Scripts\python.exe manage.py seed_demo
Run all tests	venv\Scripts\python.exe manage.py test
Set up real email (Gmail app password)	.\scripts\setup_email.ps1

The scripts call `venv\Scripts\python.exe` directly, so the project's own packages are always used even if you forget to activate the virtual environment.

12.2 Where to change things

I want to change...	Edit
EMD amount, anti-snipe window, notify losers, worker interval ...	.env (names listed in .env.example)
The words in an email	*_subject.txt / *_body.txt in the app's templates/.../emails/ folder
What a page looks like	The HTML template, and static/css/site.css
A bid rule	auctions/bidding.py (and its test in test_bidding.py)
PDF layout	accounts/pdfs.py
Excel columns	EXPECTED_HEADERS in auctions/importers.py

`docs/HOW_TO_CHANGE.md` has the complete list.

13. Questions your mentor might ask

Q. Why Django?

A. It gives us login, password hashing, forms, admin, the ORM and security protections (CSRF, XSS escaping) built in, so we write only the auction-specific parts.

Q. Why PostgreSQL and not SQLite?

A. SQLite ignores select_for_update(), so it cannot safely handle two bids at the same instant. PostgreSQL supports row locks; the concurrency tests prove it.

Q. How do you stop two people winning with the same bid?

A. Each bid locks the auction row; bids are processed one at a time; the second sees the new highest and is rejected. As a backstop, the database refuses two bids with the same amount in one auction.

Q. What if the worker runs twice or a browser also closes the auction?

A. close_auction() is idempotent: lock, re-check status, then act. Only the first closer does anything; and the quarter_won_only_once constraint makes a double award impossible in the database.

Q. How is the bidder anonymous?

A. Every page and WebSocket message is built from realtime.py and shows only "Bidder N". The mapping to the real person is in Participation, visible to superusers only, kept for audits and to email the winner. It is anonymous to other users, not to the database.

Q. Why WebSockets?

A. With HTTP the browser would have to keep asking "any new bids?". A WebSocket lets the server push each new bid to everyone instantly.

Q. What happens if my internet drops while bidding?

A. auction_room.js reconnects automatically and the server sends a fresh snapshot, so the page shows the true highest bid and time. If a bid was sent twice, the client_bid_id makes the second one a harmless duplicate.

Q. What is anti-sniping?

A. A bid in the last 60 seconds extends the deadline (by the auction's own extension, default 2 minutes), so others can respond.

Q. Can a user cheat by editing the JavaScript?

A. No. The browser only displays. Every rule is checked on the server in bidding.py.

Q. How is someone stopped from activating another person's account?

A. Activation needs the email AND the secret URN that was emailed only to that person, the URN is random, and attempts are rate limited. Also the admin must have verified the EMD payment.

Q. Which rules are official?

A. None are confirmed as official SAIL policy. EMD amount, increment, extension, tie rule, no-bids outcome and "base rate" meaning are labelled placeholders (see PROGRESS.md). The "one quarter at a time" rule was confirmed by the project owner.

14. Glossary

Term	Meaning
ASGI	Standard interface letting Python servers handle HTTP and WebSockets.
Atomic / transaction	A group of database changes that all happen or none happen.
Channel layer	The message carrier that delivers group messages to consumers (memory or Redis).
Consumer	Handles one WebSocket connection, like a view handles one HTTP request.
Constraint	A rule the database itself enforces (unique, check).
Context processor	Function adding values to every template.
CSRF	Cross-Site Request Forgery: another site tricking your browser into submitting our forms.
Daphne	The ASGI server that runs this project.
Deadlock	Two programs each waiting for a lock the other holds; avoided by a fixed lock order.
Decorator	@something above a function, wrapping it with extra behaviour.
EMD	Earnest Money Deposit: a deposit paid before bidding.
Foreign key	A field pointing to a row in another table.
Group	Named set of WebSocket connections (one per auction).
Hashing	One-way scrambling of passwords so they can be checked but not read.
Idempotent	Doing it twice has the same effect as doing it once.
Management command	A custom "python manage.py xyz" command.
Migration	A file describing a change to database tables.
ModelForm	A form whose fields are built from a model.
on_commit	Run some code only after the transaction has been saved successfully.
ORM	Writing database queries as Python instead of SQL.
PROTECT	on_delete rule: refuse to delete a row that others point to.
Race condition	A bug where the result depends on which of two simultaneous actions runs first.
Rate limiting	Refusing further attempts after too many failures in a time window.
Redis	Fast in-memory data store, used as shared channel layer and cache in production.
Row lock / select_for_update	Making other transactions wait before touching the same row.
Snapshot	The complete current state of an auction sent on connect/reconnect.
URN	Unique Reference Number: random 7-digit number; later the login username.
UUID	A random 128-bit id, used here as client_bid_id.
WebSocket	A connection that stays open so both sides can send messages at any time.
XSS	Cross-Site Scripting: injecting script into a page; prevented by escaping.